

## CO5 AT2

[← Back](#)

## QUESTIONS

Q1

Secure Employee Record  
Management System  
50 marks

Q2

Bank Transaction File and  
Database System  
50 marks

2/2 answered

## Q1 Secure Employee Record Management System

50 marks

Develop a Java application for an organization to securely manage employee records using hashing and an appropriate hash function for employee identification. Store employee details in a file using byte/character streams and implement serialization for storing employee objects. Provide options to add, search, update, and delete employee records. The system should handle duplicate IDs and invalid file operations appropriately.

## Your Code

```
1 import java.util.*;
2 import java.security.MessageDigest;
3
4 public class Employee {
5     String id;
6     String name;
7     String department;
8     double salary;
9
10    Employee(String id, String name, String department, double salary)
11    {
12        this.id = id;
13        this.name = name;
14        this.department = department;
15        this.salary = salary;
16    }
17
18    static String hash(String id) throws Exception {
19        MessageDigest md = MessageDigest.getInstance("SHA-256");
20        byte[] bytes = md.digest(id.getBytes("UTF-8"));
21
22        StringBuilder result = new StringBuilder();
23
24        for (byte b : bytes) {
25            result.append(String.format("%02x", b));
26        }
27    }
28 }
```

## Input

```
101
Rahul
IT
50000
```

## Output

```
Employee added successfully
ID: 101
Name: Rahul
Department: IT
Salary: 50000.0
```

## CO5 AT2

[← Back](#)

## QUESTIONS

Q1

Secure Employee Record  
Management System  
50 marks

Q2

Bank Transaction File and  
Database System  
50 marks

2/2 answered

## Q2 Bank Transaction File and Database System

50 marks

Design a banking application that maintains customer and transaction information using Java I/O and JDBC. Store transaction logs using stream I/O and serialize customer objects for backup. Establish JDBC connectivity with a relational database and implement INSERT, UPDATE, DELETE, and SELECT operations using appropriate Statement types. The application must validate transactions and handle database and I/O exceptions.

## Your Code

```
1 import java.io.*;
2 import java.util.*;
3
4 public class BankTransaction implements Serializable {
5
6     static class Customer implements Serializable {
7         int id;
8         String name;
9         double balance;
10
11         Customer(int id, String name, double balance) {
12             this.id = id;
13             this.name = name;
14             this.balance = balance;
15         }
16     }
17
18     static ArrayList<Customer> customers = new ArrayList<>();
19
20     static void insert(int id, String name, double balance) {
21         for (Customer c : customers) {
22             if (c.id == id) {
23                 System.out.println("Duplicate customer id");
```

## Input

```
1
101
Rahul
50000
```

## Output

```
Customer inserted
```